

11RocketFuel-AB-test

October 5, 2021

1 Data Science: Bridging Principles and Practice

1.1 Part 11: The A/B Test (Rocket Fuel case study)

1.1.1 Table of Contents

11 - Section ??

a - Section ??

b - Section ??

c - Section ??

In [1]: *# load some necessary software: THIS CELL MUST BE RUN*

```
import pandas as pd
import numpy as np
import math
import matplotlib.pyplot as plt
%matplotlib inline
from scipy.stats import ttest_ind
import seaborn as sns
from numpy.random import permutation
from gofer.ok import check
```

```
# set the random seed for reproducibility
np.random.seed(28)
```

In [2]: *# load the Rocket Fuel data*

```
ads = pd.read_csv('data/rocketfuel_data_renamed.csv', index_col=0)
```

```
# display the first ten rows
ads.head()
```

```
Out[2]:
```

	user id	test group	converted	total ads	most ads	ads day	most ads	hour
0	1069124	ad	False	130		Monday		20
1	1119715	ad	False	93		Tuesday		22
2	1144181	ad	False	21		Tuesday		18
3	1435133	ad	False	355		Tuesday		10
4	1015700	ad	False	276		Friday		14

1.2 11. The Rocket Fuel A/B Test

Rocket Fuel's initial goal was to say whether or not the advertising campaign they ran for TaskBella was effective. In Notebook 10, we started to answer this question by calculating the *difference in average rates of conversion* between the control (PSA) group and the test (ad) group. However, we don't yet know if that difference is *statistically significant*.

This notebook will show how to calculate measures of statistical significance in Python, including how to:

- use **randomization** for hypothesis testing
- run a **permutation test** to see if the observed results of an A/B test are significant
- calculate the **p-value** and **standard error**

The code to calculate the difference is replicated below, in compressed form. Run next two cells to output the two rates and the difference between them.

```
In [3]: # a DataFrame with the conversion rates for the ad and psa groups
        avgs_by_group = ads.groupby("test group").mean()

        # an array with the two conversion rates
        observed_rates = avgs_by_group["converted"]

        # the difference between the average ad and psa rates
        observed_diff = observed_rates[0] - observed_rates[1]
        observed_diff
```

```
Out [3]: 0.007692453192201517
```

We can see that there is a difference, but that difference is very small. What we now what to know is whether that difference is *significant*. Can the difference between the two rates be explained by chance variation? Or, is the difference too large to be likely under chance variation, providing evidence that the ad campaign was effective in increasing bag purchases?

Our hypotheses are as follows:

- **Null hypothesis:** users who saw ads were just as likely to purchase a handbag as users who saw PSAs. Any difference in the conversion rates seen in the sample data is due to random chance.
- **Alternative hypothesis:** users who saw ads were more likely to buy handbags than users who saw PSAs.

1.2.1 11a. Permutation

If we assume the null hypothesis, then any user had the same chance of converting as any other, regardless of whether or not they saw the ad. This means that if we took all the values from the "converted" column and randomly reassigned them to all the users in the study (both control and experimental group members), we should see a similar difference in conversion rates between the two groups.

The permutation function from the Numpy library can help us out with the random reassignment. Let's look at an array of the numbers 1 through 10.

```
In [4]: # a DataFrame of numbers
numbers = np.arange(1, 11)
numbers
```

```
Out[4]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

permutation takes the values in an array, randomly shuffles them, and returns a new array of the same length with the values in a different order. Run the next cell a few times to see a few different permutations.

```
In [5]: # get a random permutation of the numbers array
permutation(numbers)
```

```
Out[5]: array([10,  3,  9,  8,  4,  1,  5,  7,  6,  2])
```

Note that this is equivalent to **sampling without replacement**: picking the numbers 1-10 out of a metaphorical hat one-by-one, where each number is as likely as any other number to be chosen, but once a number is chosen ("pulled out of the hat") it will not be chosen again.

Now we can permute the values in our "converted" column.

```
In [6]: # select the "converted" column.
converted = ads["converted"]

# permute the "converted" column
converted_permutation = permutation(converted)
converted_permutation
```

```
Out[6]: array([False, False, False, ..., False, False, False])
```

We then want to attach this column of shuffled values to our original ads DataFrame.

```
In [7]: # add a new column with the permutation
ads["converted permutation"] = converted_permutation
ads.head()
```

```
Out[7]:
```

	user id	test group	converted	total ads	most ads day	most ads hour	\
0	1069124	ad	False	130	Monday	20	
1	1119715	ad	False	93	Tuesday	22	
2	1144181	ad	False	21	Tuesday	18	
3	1435133	ad	False	355	Tuesday	10	
4	1015700	ad	False	276	Friday	14	

	converted permutation
0	False
1	False
2	False
3	False
4	False

Then, we group by "test group" again to see the conversion rates by group for the permutation.

```
In [8]: # group by test group
        with_perm = ads.groupby("test group").mean()

        perm_rates = with_perm["converted permutation"]
        perm_rates
```

```
Out[8]: test group
ad      0.025231
psa     0.025421
Name: converted permutation, dtype: float64
```

And finally, we calculate the difference between the "ad" and "psa" group conversion rates for the permutation.

```
In [9]: # the difference between the permutation ad and psa group rates
        perm_rate_diff = perm_rates[0] - perm_rates[1]

        # print the original rate difference and the permutation rate difference
        print("Observed conversion rate difference: ", observed_diff,
              "\nPermutation conversion rate difference: ", perm_rate_diff)
```

```
Observed conversion rate difference:  0.007692453192201517
Permutation conversion rate difference: -0.00018956744757011798
```

QUESTION: How does the difference in conversion rates for the permuted data compare to the original data?

ANSWER:

This probably implies that the conversion rate is significant for the ads group and that if the we keep the overall conversion rate the same assuming ads did not play a factor then conversion rate difference is almost zero. But that is not what happened.

1.2.2 11b. Simulation

Here we run into a problem similar to our original issue: how do we know that the difference seen in this new permutation is *also* not due to random chance?

One way is to repeat the process from section 11a- randomly permute the conversion data, attach it to the test group labels in their original order, and calculate the test statistic- many, many times. This will give us the **empirical distribution** of values for the **test statistic** (the difference between the two conversion rates).

To simplify things, we've made a function that does all of code in part 11a. `calc_perm_rate_diff` randomly shuffles the values in "converted", groups them by test group, and returns the difference between the conversion rates for the two groups. Try running the cell a few different times to see what values it calculates.

```
In [10]: # calculate the test statistic for a random permutation
         def calc_perm_rate_diff():
             """Randomly permute the conversion data and return
```

```
the difference in conversion rates between the control and
experimental groups."""
```

```
perm = np.random.permutation(ads["converted"])
test_pd = ads["test_group"]
control = perm[test_pd == "psa"]
exper = perm[test_pd == "ad"]
return np.mean(exper) - np.mean(control)
```

```
calc_perm_rate_diff()
```

```
Out[10]: -0.0005438155662115425
```

For Loops We want to calculate the test statistic many times, but ideally we want to do so without copying and pasting calls to `calc_perm_rate_diff` thousands of times.

Thankfully Python has a statement called a **for loop** that allows us to repeat an action or set of actions over and over again. Here's an example:

```
In [11]: # a for loop
        for i in range(5):
            print("Hi there!")
```

```
Hi there!
Hi there!
Hi there!
Hi there!
Hi there!
```

We can break down a for loop statement (i.e. the second line in the above cell) into four parts:

- The word "for"
- A name we want to give each item in a sequence
- The word "in"
- A sequence (e.g. "range(5)" to go through numbers 0 up to but not including 5)

The for loop statement tells the computer to do all of the indented code below the statement (also called the code "inside" the loop), for each item in the sequence. So, if we want to calculate the test statistic for 1000 permutations of the "converted" data, we can put our calculation function inside a for loop.

We want to be careful: unless we save the calculations somewhere outside the loop, the computer will just overwrite them each time it goes through the loop. The next cell creates an empty array called `diff_array` to store the differences we calculate and adds an extra command inside the for loop to add each difference to the list after it's calculated.

Run the next cell to calculate the test statistic for 1,000 permutations of the data.

Note: this cell may take up to a minute to finish running (remember- the computer is working with almost 600,000 rows, and it's doing it 1,000 times).

```
In [12]: n = 1000
        diff_array = np.empty(n)
```

```

for i in range(n):
    # calculate the rate difference
    diff = calc_perm_rate_diff()

    # add the difference to the array
    diff_array[i] = diff

    # code to periodically print out a status update
    if i % 100 == 0:
        print("{}% done".format(i/10))
    elif i == 999:
        print("done!")

```

```

0.0% done
10.0% done
20.0% done
30.0% done
40.0% done
50.0% done
60.0% done
70.0% done
80.0% done
90.0% done
done!

```

Now, let's put all the test statistics we calculated in a DataFrame.

```

In [13]: # DataFrame of calculated test statistics
differences = pd.DataFrame({"rate difference": diff_array})
differences.head()

```

```

Out[13]:   rate difference
0      -0.000455
1       0.000298
2       0.000342
3       0.001405
4      -0.000942

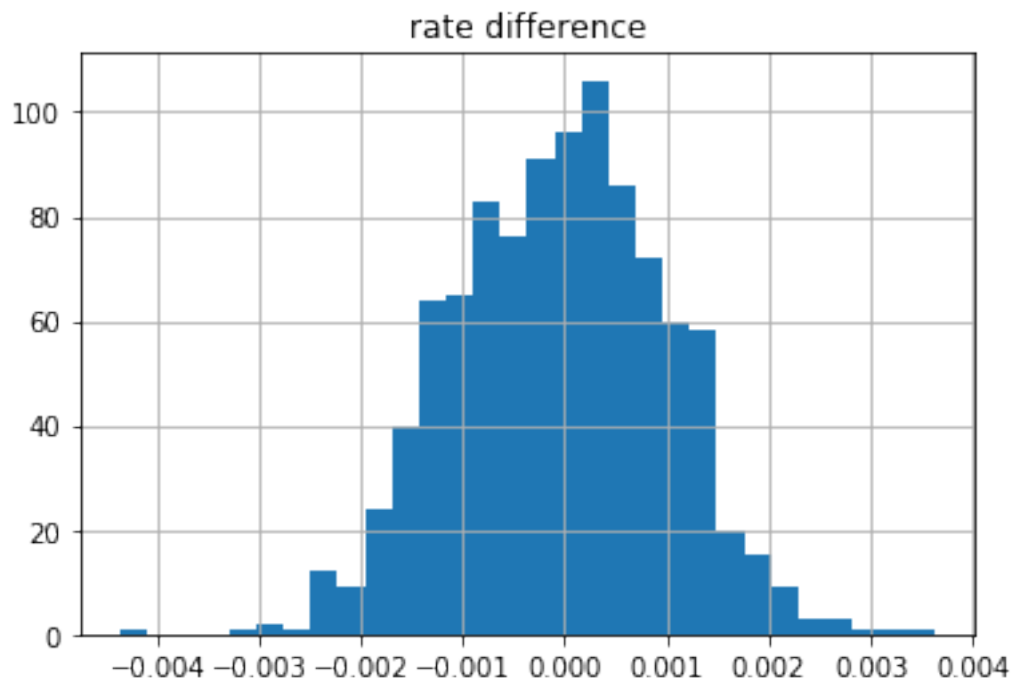
```

And finally, we'll use `.hist` to visualize the empirical distribution of the test statistic in a histogram. We've increased the optional `bins` argument from the default value to see a bit more detail.

```

In [14]: # the empirical distribution of the test statistic
differences.hist("rate difference", bins=30);

```



We see that the distribution is roughly normal and centered around zero, which makes sense since our null hypothesis assumes that the control and experimental group conversion rates are the same, and the difference between two equal rates is zero.

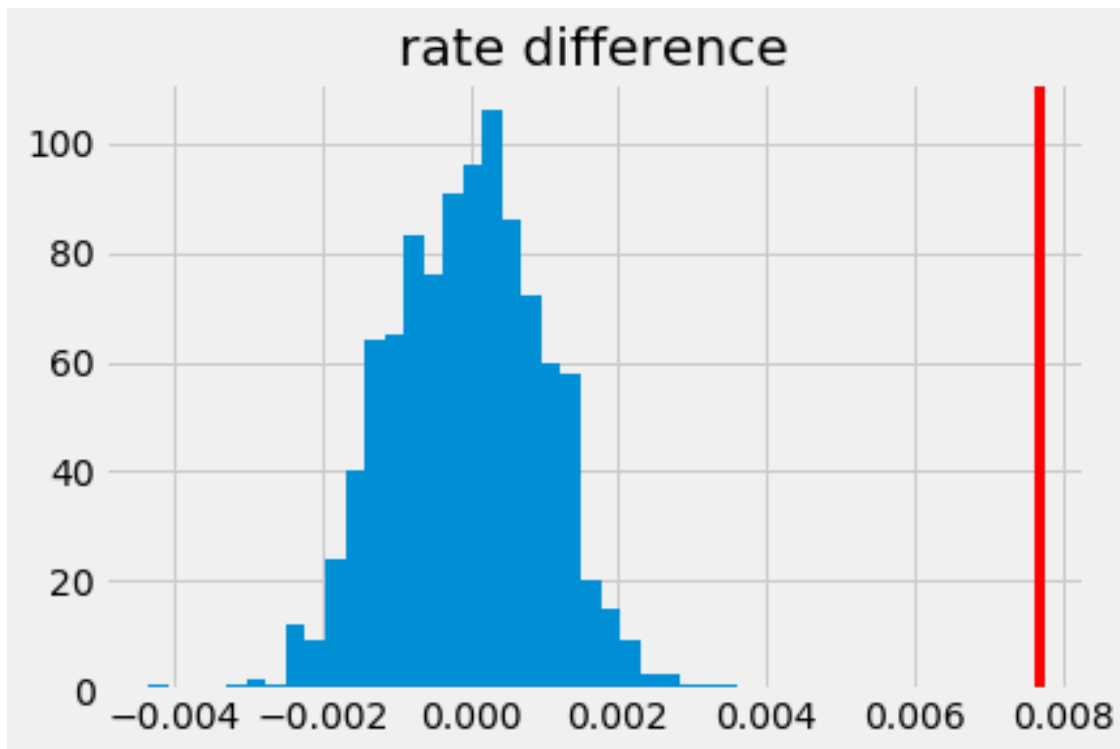
Now, let's see where the rate difference we observed in the original sample falls compared to these rates. The following cell plots the histogram again and adds a vertical line at the value of the test statistic in the original sample.

```
In [15]: # plot the empirical histogram
plt.style.use("fivethirtyeight")

differences.hist("rate difference", bins=30);

# plot the original sample statistic
plt.axvline(x=observed_diff, color="r")
```

```
Out[15]: <matplotlib.lines.Line2D at 0x7f834ae67358>
```



1.2.3 11c. P-Values, T-Tests, and Standard Error

Our original observed difference looks like it falls way outside all the values in the empirical distribution. We can also note that our empirical distribution looks roughly normal- this makes sense by the Central Limit Theorem because our test statistic is an average.

We can get a measure of exactly how far outside it is by calculating the p-value. Remember, the p-value in this case is the chance that, under the null hypothesis, the average difference in conversion rates is the *as large or larger* than the difference we saw in our original sample.

Two-Tailed vs One-Tailed P Values

We are calculating the one-tailed p value: the chance that a rate difference average un

EXERCISE: Calculate the empirical p-value.

Create a DataFrame <code>more_than_original</code> that contains all the rows of <code>

Get the count of items in the <code>more_than_original</code> DataFrame and the count o

Divide <code>more_than_count</code> by <code>total_diffs_count</code> to get the empir

</div>

```
In [16]: # A DataFrame of values in the empirical distribution that are at least as large as
         more_than_original = differences[differences["rate difference"] >=observed_diff]
```

```
         # the number of rows in more_than_original
```

```

more_than_count = more_than_original.shape[0]
# the number of rows in differences
total_diffs_count = differences.shape[0]

# the empirical p value
empirical_p = more_than_count/total_diffs_count
empirical_p

```

Out[16]: 0.0

```

In [17]: # run this cell to check your answer for some common errors
check("tests/empirical-p.ok")

```

Out[17]: <gofer.ok.OKTestsResult at 0x7f834ad97dd8>

We can check our work with a pre-made function, as well. Because it is a common operation, the **Scipy** Python library for math, science, and engineering has a function that calculates the T-test for two independent samples. `ttest_ind` takes two arguments- the two arrays containing the samples from the two different groups- and returns the calculated T statistic and the two-sided p value.

The **T statistic** is similar to the **z score**, but used when the population parameters (like the standard deviation) are unknown.

Since our test statistic is normally distributed, the one-tailed p value should be half of the two-tailed.

Note that very small values will be returned in *scientific notation*, specifically *e-notation*.

```

In [18]: # divide our DataFrame into experiment and control groups
exper_group = ads[ads["test group"] == "ad"]
control_group = ads[ads["test group"] == "psa"]

# get the conversion data as arrays
exper_conversion = exper_group["converted"]
control_conversion = control_group["converted"]

# calculate the T-statistic and P-value for the
ttest_ind(exper_group["converted"], control_group["converted"])

```

Out[18]: Ttest_indResult(statistic=7.37040597428566, pvalue=1.7033052627831264e-13)

QUESTION: A statistically significant p value is conventionally defined as less than

ANSWER: The p-value is not significant and is less than the sensitivity of either 5% or 1% and so can reject the null hypothesis. We should check based on the cost of the ad campaign, whether this results in a positive roi before proceeding with the ad campaign

References

- Rocket Fuel data and discussion questions adapted from materials by Zsolt Katona and Brian Bell, BerkeleyHaas Case Series
- "For loop" section adapted from materials by Kelly Chen at [UC Berkeley Data Modules](#)

Author: Keeley Takimoto